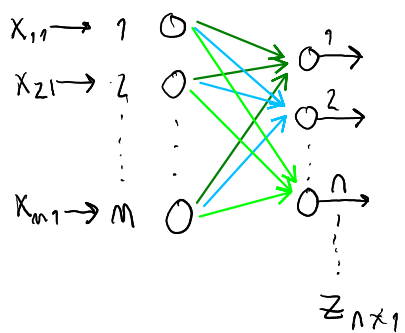




pesos * input + biases = z_{nx1}

$$W_{n \times m} X_{m \times 1} + b_{n \times 1} = z_{n \times 1} \quad (3.9)$$

Usando un batch para entrenar con p ejemplos a la vez:

$$W_{n \times m} X_{m \times p} + \underbrace{[b_{n \times 1} \dots b_{n \times 1}]}_{\text{Broadcast}} = z_{n \times p}$$

$$W_{n \times m} I_{m \times p} + \underbrace{b_{n \times p}}_{\text{batch}} = z_{n \times p}$$

p predicciones a la vez

Función de activación softmax a la salida: $\rightarrow$ probabilidad

$$\hat{y}_k = P(Y=k) = \frac{e^{z_{k1}}}{\sum_j e^{z_{j1}}} \quad (4.1)$$

$\rightarrow k=1, \dots, n$

$\rightarrow j=1, \dots, n$

Usando batch: (4.2)

$$\hat{y}_{ki} = P(Y=k | X=x_i) = \frac{e^{z_{ki}}}{\sum_j e^{z_{ji}}}$$

$\rightarrow k=1, \dots, n$

$\rightarrow i=1, \dots, p$

$\rightarrow j=1, \dots, n$

$$\hat{y} = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

$$\sum_k \hat{y}_{ki} = \frac{\sum_k e^{z_{ki}}}{\sum_j e^{z_{ji}}} = 1 \quad \checkmark$$

$\hat{y}_{ki}$ más alto = predicción del ejemplo i.

Loss Función para el ejemplo i = Cross entropy = Xentropy

$$L_i = - \sum_k \underbrace{y_{ki}}_{\substack{\text{valor correcto} \\ \text{de la salida}}} \ln(\hat{y}_{ki}) \quad (5.1)$$

y_{ki} son los elementos de un one-hot vector:

$$y_{ki} = 0, 0, \dots, 0, 1, 0, \dots, 0$$

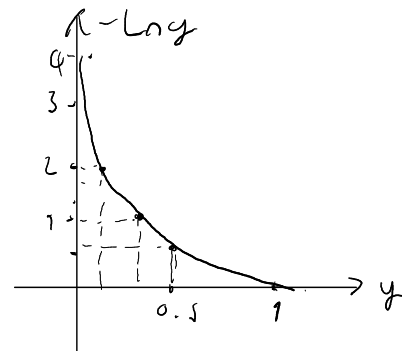
$\uparrow$
k element
 $\downarrow$
clase esperada (correcta)

$$y_{ki} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \leftarrow k \text{ element}$$

$\leftarrow n \text{ element}$

$$\begin{aligned} L_i &= -\ln(\hat{y}_{ki}) \\ &= -\ln\left(\frac{e^{z_{ki}}}{\sum_j e^{z_{ji}}}\right) \quad (5.2) \end{aligned}$$

De la salida es pérdida para el i



Cost function para los pesos, biases del batch:

$$J(W, b) = \frac{1}{p} \sum_i L_i \quad (5.3)$$

$$J(W, b) = \frac{1}{p} \sum_i -\ln\left(\frac{e^{z_{ki}}}{\sum_j e^{z_{ji}}}\right)$$

$\rightarrow i=1, \dots, p$

Para una sola neurona, una sola entrada:

$$x \rightarrow \text{O} \xrightarrow{w} \text{O} \rightarrow z$$

$$wx + b = z \quad (6.1)$$

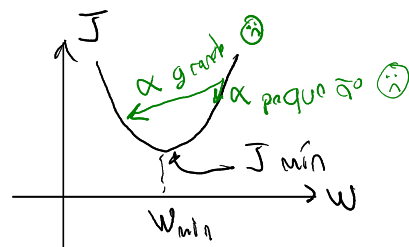
$$J(w, b)$$

Gráfica computacional: Cada operación es un nodo



Gradient descent:

$$\rightarrow \frac{\partial J}{\partial w} = \lim_{h \rightarrow 0} \frac{J(w+h) - J(w)}{h} \quad (6.2)$$



$$w = w - \alpha \frac{\partial J}{\partial w} \quad (6.3)$$

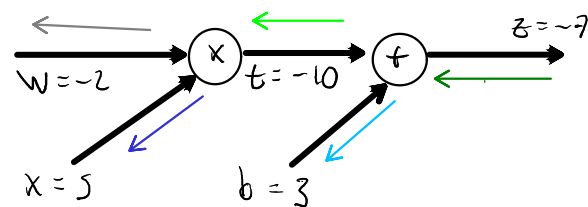
learning rate
= stop size

De igual manera para b:

$$b = b - \alpha \frac{\partial J}{\partial b} \quad (6.4)$$

Backpropagation:

Gráfica computacional:



$$\rightarrow z = t + b$$

$$z = wx + b$$

Como es la última (y única) capa de la NN:

$$\frac{dz}{dz} = 1 \quad (7.1)$$

$$\rightarrow \frac{dz}{db} = \frac{dz}{dz} \frac{dz}{db} = 1$$

$$\rightarrow \frac{dz}{dt} = \frac{dz}{dz} \frac{dz}{dt} = 1$$

$$\rightarrow \frac{dz}{dx} = \frac{dz}{dt} \frac{dt}{dx} = w = -2 \quad (7.2)$$

$$\rightarrow \frac{dz}{dw} = \frac{dz}{dt} \frac{dt}{dw} = x = 5 \quad (7.3)$$

$$\text{Ejemplo: } h = 0.1$$

cambio de w

$$\rightarrow w = w_0 + h = -2 + 0.1 = -1.9$$

$$\rightarrow t = wx_0 = (-1.9)(5) = -9.5$$

$$\rightarrow z = t + b_0 = (-9.5) + (3) = -6.5$$

$$\rightarrow z = z_0 + h \frac{dz}{dw}$$

$$z = -7 + (0.1) 5$$

$$z = -6.5 \checkmark$$

Ej m: $h=0.1$

↑
cambio de x

→ $x = x_0 + h = 5 + 0.1 = 5.1$
 → $t = w_0 x = (-2)(5.1) = -10.2$
 → $z = t + b_0 = (-10.2) + (3) = -7.2$

→ $z = z_0 + h \frac{dz}{dx}$

$z = -7 + (0.1)(-2)$

$z = -7.2 \checkmark$

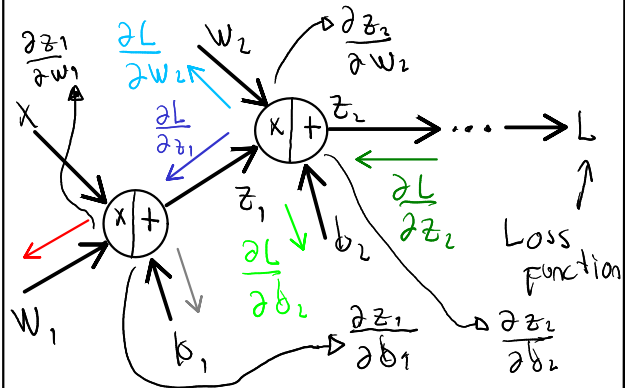
Ej m: $h=0.1$

↑
cambio de b :

→ $b = b_0 + h = 3 + (0.1) = 3.1$
 → $z = t_0 + b = (-10) + (3.1) = -6.9$
 → $z = z_0 + h \frac{dz}{db}$

$z = -7 + (0.1)(1) = -6.9 \checkmark$

Para dos perceptrons:
Gráfica computacional



$\frac{\partial L}{\partial w_2} = ?$ $\frac{\partial L}{\partial b_2} = ?$ $\frac{\partial L}{\partial w_1} = ?$ $\frac{\partial L}{\partial b_1} = ?$

→ $\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial b_2}$ → $b_2 = b_2 - \alpha \frac{\partial L}{\partial b_2}$

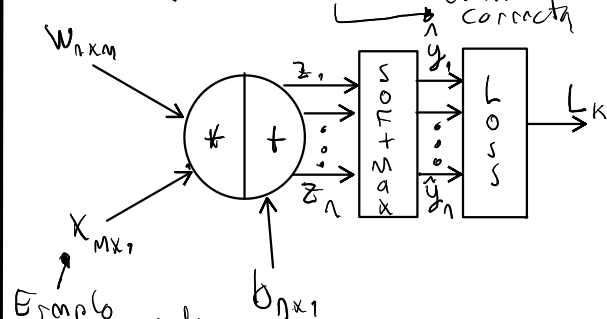
→ $\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial w_2}$ → $w_2 = w_2 - \alpha \frac{\partial L}{\partial w_2}$

→ $\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial z_1}$

→ $\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial b_1}$ → $b_1 = b_1 - \alpha \frac{\partial L}{\partial b_1}$

→ $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial w_1}$ → $w_1 = w_1 - \alpha \frac{\partial L}{\partial w_1}$

Para un solo ejemplo etiquetado como clase k :



Ejemplo etiquetado

$w_{n \times m} x_{m \times 1} + b_{n \times 1} = z_{n \times 1}$

→ $L = - \sum_k y_k \ln \hat{y}_k$ (S.1)

$\hat{y} = (0 \ 0 \ \dots \ 1 \ \dots \ 0)$
↑
 k elemento

$L = - \ln \hat{y}_k = - \ln \left(\frac{e^{z_k}}{\sum_j e^{z_j}} \right)$
↑
De la salida es permitida para el ejemplo

$j = 1, \dots, n$

$L = - \ln \left(\frac{e^{z_k}}{\sum_j e^{z_j}} \right) = \ln \sum_j e^{z_j} - z_k$

→ $\frac{\partial L}{\partial z_i} = ?$

$\frac{\partial L}{\partial z_i} = \frac{\partial \ln \left(\sum_j e^{z_j} \right)}{\partial z_i} - \frac{\partial z_k}{\partial z_i}$

$= \frac{1}{\sum_j e^{z_j}} \frac{\partial \sum_j e^{z_j}}{\partial z_i} - \delta_{ik}$

$= \frac{e^{z_i}}{\sum_j e^{z_j}} - \delta_{ik}$
↑
Delta de Kronecker

$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_k$ (10.1)

→ Predicción para la clase i
Para el score de la salida esperada (clase correcta):

$\frac{\partial L}{\partial z_k} = \hat{y}_k - y_k$ (10.2)

Para inicializar W y b en una NN con pocas capas se puede!

$$\rightarrow W_{n \times m} = n.p. \text{ random, random } (n, m) * 0.01 \quad (11.1)$$

$n \times m$

$$\rightarrow b_{n \times 1} = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix} = n.p. \text{ zeros } ((n, 1)) \quad (11.2)$$

$$\rightarrow z = W @ x + b \quad (11.3)$$

$\frac{\partial L}{\partial z}$ en forma matricial: $\leftarrow (10.2)$

$$\frac{\partial L}{\partial z_{n \times 1}} = \hat{y}_{n \times 1} - \begin{pmatrix} 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix}_{n \times 1} \quad (11.4)$$

← k elementos
↑
etiquetas de cada ejemplo

one-hot vector = $y_{n \times 1}$ (green)

$$\rightarrow \frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial W} = \frac{\partial L}{\partial z} x$$

$$\frac{\partial L}{\partial W_{n \times m}} = \frac{\partial L}{\partial z_{n \times 1}} \cdot (X_{m \times 1})^T \quad (11.5)$$

$$\frac{\partial L}{\partial W_{n \times m}} = (\hat{y}_{n \times 1} - y_{n \times 1}) \cdot (X_{m \times 1})^T \quad (11.6)$$

$$= (y_{\text{hat}} - y) @ X.T$$

$$\rightarrow \frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial b} = \frac{\partial L}{\partial z} (1) \quad (11.7)$$

$$\frac{\partial L}{\partial b_{n \times 1}} = \frac{\partial L}{\partial z_{n \times 1}} \cdot (1)_{1 \times 1} = \frac{\partial L}{\partial z_{n \times 1}}$$

$$= \hat{y}_{n \times 1} - y_{n \times 1} \quad (11.7)$$

Después se puede actualizar W y b :

$$\rightarrow W_{n \times m} = W_{n \times m} - \alpha \frac{\partial L}{\partial W_{n \times m}} \quad (11.8)$$

Bj,m: 0.01

$$\rightarrow b_{n \times 1} = b_{n \times 1} - \alpha \frac{\partial L}{\partial b_{n \times 1}} \quad (11.9)$$

↑
(6.4)

For a batch with r examples
 $W_{n \times m} X_{m \times r} + b_{n \times 1} = z_{n \times r}$

Loss functions of r examples

$$L_{1 \times r} = L_r = -\ln \left[\frac{(e^{z_{ki}})_{1 \times r}}{\left(\sum_j e^{z_{ji}} \right)_{1 \times r}} \right]$$

De la salida esperada para el ejemplo

using Jacobian matrix:

$$\frac{\partial L_r}{\partial z_n} = \begin{pmatrix} \frac{\partial L_1}{\partial z_1^1} & \dots & \frac{\partial L_1}{\partial z_n^1} \\ \vdots & \ddots & \vdots \\ \frac{\partial L_r}{\partial z_1^r} & \dots & \frac{\partial L_r}{\partial z_n^r} \end{pmatrix} = \frac{\partial L}{\partial z_{r \times n}}$$

For row i (example i) of r examples:

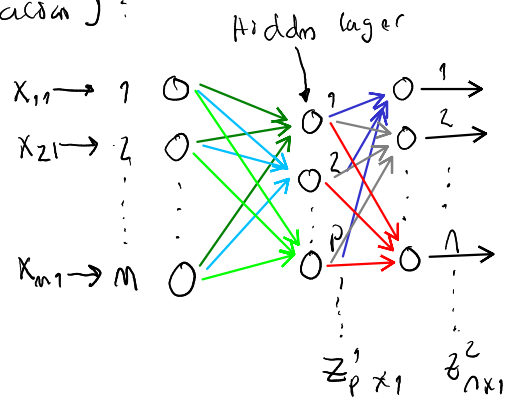
$$\frac{\partial L}{\partial z_{i \times n}} = \hat{y}_{i \times n} - y_{i \times n}$$

$$\rightarrow \frac{\partial L}{\partial z_{r \times n}} = \hat{y}_{r \times n} - y_{r \times n}$$

one-hot vectors!

$$\rightarrow \begin{pmatrix} 0 & 1 & \dots & 0 \\ 1 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix}$$

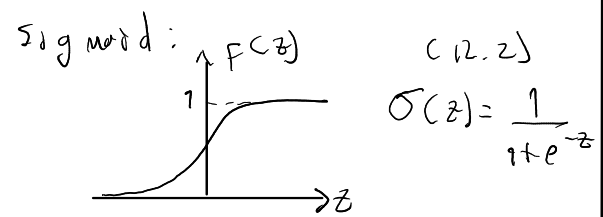
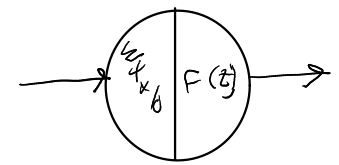
Deep learning con dos capas y solo funciones lineales de activación (sin funciones de activación):



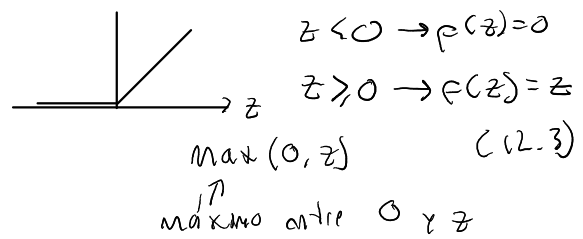
$$\begin{aligned} \rightarrow z'_{px1} &= W^1_{p \times m} x_{mx1} + b^1_{px1} \\ \rightarrow z^z_{nx1} &= W^2_{n \times p} z'_{px1} + b^2_{nx1} \\ &= W^2_{n \times p} (W^1_{p \times m} x_{mx1} + b^1_{px1}) + b^2_{nx1} \\ &= (W^2 W^1)_{n \times m} x_{mx1} + (W^2 b^1)_{nx1} + b^2_{nx1} \\ &= (W^2 W^1)_{n \times m} x_{mx1} + (W^2 b^1 + b^2)_{nx1} \end{aligned} \quad (12.1)$$

Entonces las hidden layer no activan, dando igual usar la sola

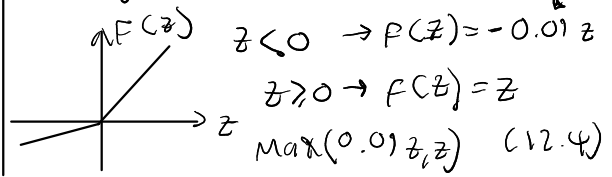
capa de salida con n perceptrones
Funciones de activación:



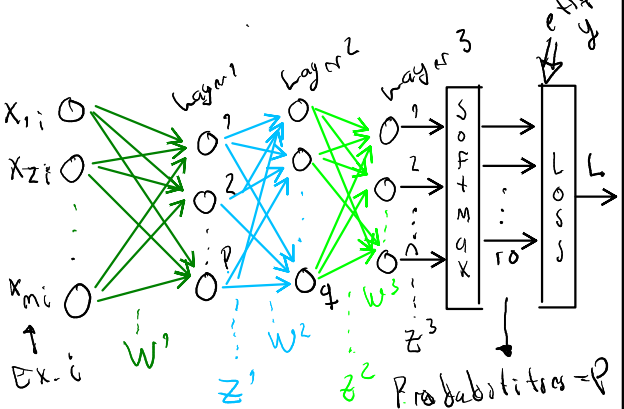
ReLU: Rectified Linear Unit:



Leaky ReLU:



Usando funciones de activación con los hidden layers:



R : Ejemplos totales
 Usando un Batch de r ejemplos:
 $\frac{R}{r}$ = pasados para entrenar con los R elementos
 Forwardpass
 Backpropagation

$\rightarrow X_{m \times r}$ Pixels
 Batch
 $\rightarrow z^1_{p \times r} \rightarrow a^1_{p \times r}$
 $\rightarrow z^2_{q \times r} \rightarrow a^2_{q \times r}$
 $\rightarrow z^3_{n \times r}$
 $\rightarrow P_{n \times r}$
 $\rightarrow L_{1 \times r} \rightarrow \text{cost}$ scalar.
 Training

Forwardpass:
 Broadcast.
 $z^1_{p \times r} = W^1_{p \times m} X_{m \times r} + (b^1_{p \times 1} \dots b^1_{p \times 1})_{p \times r}$
 Después de aplicar la función de activación de la capa 1:
 $a^1_{p \times r} = F^1(z^1_{p \times r})$
 $z^2_{q \times r} = W^2_{q \times p} a^1_{p \times r} + (b^2_{q \times 1} \dots b^2_{q \times 1})_{q \times r}$
 $a^2_{q \times r} = F^2(z^2_{q \times r})$

$\rightarrow z^3_{n \times r} = W^3_{n \times q} a^2_{q \times r} + b^3_{n \times r}$ Broadcast
 (11.4)
 Softmax:
 $P_{n \times r} = \frac{(e^{z^3})_{n \times r}}{\left(\sum_i e^{z^3_{i,j}} \right)_{1 \times r}} = y_{n \times r}$ (4.7)
 suma cada columna (ejm) de $(e^{z^3})_{n \times r}$

Cross entropy:
 $\rightarrow L_{1 \times r} = -\ln(P_{a1} P_{b2} \dots P_{zn})_{1 \times r}$
 etiquetas $a_j=1$, $a_j=2$
 $\rightarrow \text{cost} = \sum_i L_{1 \times r}$
 (5.3)

Backpropagation:
 $\rightarrow \frac{\partial L}{\partial z^3_{n \times r}} = P_{n \times r} - y_{n \times r}$ etiquetas
 one hot vector per cada ejemplo:
 $\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 1 \end{pmatrix} \leftarrow \begin{matrix} 1 \\ 2 \\ \vdots \\ n \end{matrix}$
 $\rightarrow \frac{\partial L}{\partial a^1} = \frac{\partial L}{\partial z^3} \frac{\partial z^3}{\partial a^1}$
 $\frac{\partial L}{\partial a^2_{q \times r}} = (W^3_{n \times q})^T \frac{\partial L}{\partial z^3_{n \times r}}$ most $b^3_{n \times r}$ normalized
 $\rightarrow \frac{\partial L}{\partial W^3} = \frac{\partial L}{\partial z^3} \frac{\partial z^3}{\partial W^3}$
 $\frac{\partial L}{\partial W^3_{n \times q}} = \frac{\partial L}{\partial z^3_{n \times r}} (a^2_{q \times r})^T$ normalized

$$\rightarrow \frac{\partial L}{\partial b^3} = \frac{\partial L}{\partial z^3} \frac{\partial z^3}{\partial b^3} \quad \text{I}$$

$$\frac{\partial L}{\partial b_{n \times 1}^3} = \begin{pmatrix} \sum_j \frac{\partial L}{\partial z_{1j}^3} \\ \vdots \\ \sum_j \frac{\partial L}{\partial z_{nj}^3} \end{pmatrix}_{n \times 1}$$

sum elements of each row

normalized

$$\rightarrow \frac{\partial L}{\partial z^2} = \frac{\partial L}{\partial a^2} \frac{\partial a^2}{\partial z^2} \quad W^3 \text{ normalized}$$

$$\frac{\partial L}{\partial z_{q \times r}^2} = \left(\frac{\partial L}{\partial a_{ij}^2} \cdot \frac{\partial f^2(z_{ij}^2)}{\partial z^2} \right)_{q \times r}$$

$$= \frac{\partial f^2(z^2)}{\partial z^2} = g(z^2)$$

$g(z_{ij}^2)$

$$\rightarrow \frac{\partial L}{\partial a^1} = \frac{\partial L}{\partial z^1} \frac{\partial z^1}{\partial a^1}$$

$$\frac{\partial L}{\partial a_{p \times r}^1} = \left(W_{q \times p}^2 \right)^T \frac{\partial L}{\partial z_{q \times r}^1} \quad \text{most be normalized}$$

$$\rightarrow \frac{\partial L}{\partial W^2} = \frac{\partial L}{\partial z^2} \frac{\partial z^2}{\partial W^2}$$

$$\frac{\partial L}{\partial W_{q \times p}^2} = \frac{\partial L}{\partial z_{q \times r}^2} (a_{p \times r}^1)^T$$

$$\frac{\partial L}{\partial z^2} \leftarrow \frac{\partial L}{\partial a^2}$$

W^3 must be normalized

$$\rightarrow \frac{\partial L}{\partial b^2} = \frac{\partial L}{\partial z^2} \frac{\partial z^2}{\partial b^2} \quad \text{I}$$

$$\frac{\partial L}{\partial b_{q \times 1}^2} = \begin{pmatrix} \sum_j \frac{\partial L}{\partial z_{1j}^2} \\ \vdots \\ \sum_j \frac{\partial L}{\partial z_{qj}^2} \end{pmatrix}_{q \times 1}$$

$$\rightarrow \frac{\partial L}{\partial z^1} = \frac{\partial L}{\partial a^1} \frac{\partial a^1}{\partial z^1} \quad W^2 \text{ normalized}$$

$$\frac{\partial L}{\partial z_{p \times r}^1} = \left(\frac{\partial L}{\partial a_{ij}^1} \cdot \frac{\partial f^1(z_{ij}^1)}{\partial z^1} \right)_{p \times r}$$

$$\rightarrow \frac{\partial L}{\partial W^1} = \frac{\partial L}{\partial z^1} \frac{\partial z^1}{\partial W^1}$$

$$\frac{\partial L}{\partial W_{p \times m}^1} = \frac{\partial L}{\partial z_{p \times r}^1} (x_{m \times r})^T$$

W^2 normalized

$$\rightarrow \frac{\partial L}{\partial b^1} = \frac{\partial L}{\partial z^1} \frac{\partial z^1}{\partial b^1} \quad \text{I}$$

$$\frac{\partial L}{\partial b_{p \times 1}^1} = \begin{pmatrix} \sum_j \frac{\partial L}{\partial z_{1j}^1} \\ \vdots \\ \sum_j \frac{\partial L}{\partial z_{pj}^1} \end{pmatrix}_{p \times 1}$$

Training for each epoch, for each batch:

$$\rightarrow W'_{p \times m} = W_{p \times m}^1 - \alpha \frac{dL}{dW'_{p \times m}}$$

$$\rightarrow b'_{p \times 1} = b_{p \times 1}^1 - \alpha \frac{dL}{db'_{p \times 1}}$$

$$\rightarrow W'_{q \times p} = W_{q \times p}^2 - \alpha \frac{dL}{dW'_{q \times p}}$$

$$\rightarrow b'_{q \times 1} = b_{q \times 1}^2 - \alpha \frac{dL}{db'_{q \times 1}}$$

$$\rightarrow W_{n \times q}^3 = W_{n \times q}^2 - \alpha \frac{dL}{dW_{n \times q}^3}$$

$$\rightarrow b_{n \times 1}^3 = b_{n \times 1}^2 - \alpha \frac{dL}{db_{n \times 1}^3}$$

Accuracy using validation data:
after each epoch

Forward (validation data)

$$\text{softmax}(Z_{n \times r})$$

$$P_{n \times r}$$

$$\text{Predictions} = P_e = (P_{\max, 1}, \dots, P_{\max, r})_{n \times r}$$

max row of col 1, max row col r

$$\text{Accuracy} = \frac{\sum_{i=1}^{\text{\# batches}} \sum_{j=1}^r \delta p_{e_j} = \text{Label}_j}{\text{\# Validation data}}$$

↳ % ↳ = \# batches \times r

Prediction of a image after

epochs:

Optimal

$$\begin{cases} \rightarrow W_{p \times m}^1 \rightarrow b_{p \times 1}^1 \\ \rightarrow W_{q \times p}^2 \rightarrow b_{q \times 1}^2 \\ \rightarrow W_{n \times q}^3 \rightarrow b_{n \times 1}^3 \end{cases}$$

$$\rightarrow \text{image}_{1 \times m} = X_{1 \times m} \quad \text{Total pixels}$$

Forward ($X_{1 \times m}$)

$$Z_{n \times 1}^3$$

softmax

$$\hat{y}_{n \times 1} = P_{n \times 1}$$

$$\text{Prediction} = P_e = P_{\max} \text{ of } P_{n \times 1}$$

Usando Matriz Jacobiana:
 $\frac{\partial L_r}{\partial z_n^3} \rightarrow$ Loss function de r ejemplos
 $\rightarrow \frac{\partial L_r}{\partial z_n^3}$

$$= \begin{pmatrix} \frac{\partial L_1}{\partial z_1^3} & \dots & \frac{\partial L_1}{\partial z_n^3} \\ \vdots & & \vdots \\ \frac{\partial L_r}{\partial z_1^3} & \dots & \frac{\partial L_r}{\partial z_n^3} \end{pmatrix} = \frac{\partial L}{\partial z_{r \times n}^3}$$

$$\frac{\partial L}{\partial z_{r \times n}^3} = \hat{y}_{r \times n} - y_{r \times n}$$

one hot vectors!

$$\begin{pmatrix} 0 & 1 & \dots & 0 \\ 1 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix}$$

$$\rightarrow \frac{\partial L_r}{\partial a_q^2} = \frac{\partial L_r}{\partial z_n^3} \frac{\partial z_n^3}{\partial a_q^2}$$

$$\frac{\partial L}{\partial a_{r \times q}^2} = \frac{\partial L}{\partial z_{r \times n}^3} \frac{\partial z^3}{\partial a_{n \times q}^2} = (\hat{y}_{r \times n} - y_{r \times n}) W_{n \times q}^3$$

$$\rightarrow \frac{\partial L_r}{\partial W_{n \times q}^3} = \frac{\partial L_r}{\partial z_n^3} \frac{\partial z_n^3}{\partial W_{n \times q}^3} = (\hat{y}_{r \times n} - y_{r \times n}) a_q^2$$

Producto tensorial?

$$\rightarrow \frac{\partial L_r}{\partial b_n^3} = \frac{\partial L_r}{\partial z_n^3} \frac{\partial z_n^3}{\partial b_n^3}$$

$$\frac{\partial L}{\partial b_{r \times n}^3} = \frac{\partial L}{\partial z_{r \times n}^3} I_{n \times n} = \frac{\partial L}{\partial z_{r \times n}^3}$$

$$\rightarrow \frac{\partial L_r}{\partial W_{q \times p}^2} = \frac{\partial L_r}{\partial z_n^3} \frac{\partial z_n^3}{\partial a_q^2} \frac{\partial a_q^2}{\partial z_q^2} \frac{\partial z_q^2}{\partial W_{q \times p}^2}$$

$$= (\hat{y}_{r \times n} - y_{r \times n}) W_{n \times q}^3 \left(\frac{\partial F^2(z^2)}{\partial z^2} \right)_{q \times q} a_p^2$$

$$= \frac{\partial L}{\partial W_{r \times q \times p}^2} \rightarrow \frac{\partial L_r}{\partial z_q^2}$$

$$\rightarrow \frac{\partial L_r}{\partial b_q^2} = \frac{\partial L_r}{\partial z_n^3} \frac{\partial z_n^3}{\partial a_q^2} \frac{\partial a_q^2}{\partial z_q^2} \frac{\partial z_q^2}{\partial b_q^2}$$

$$= \frac{\partial L}{\partial b_{r \times q}^2} \hookrightarrow I_{q \times q}$$

$$\rightarrow \frac{\partial L_r}{\partial W_{p \times m}^1} = \frac{\partial L_r}{\partial z_n^3} \frac{\partial z_n^3}{\partial a_q^2} \frac{\partial a_q^2}{\partial z_q^2} \frac{\partial z_q^2}{\partial a_p^1}$$

$$\frac{\partial a_p^1}{\partial z_p^1} \frac{\partial z_p^1}{\partial W_{p \times m}^1}$$

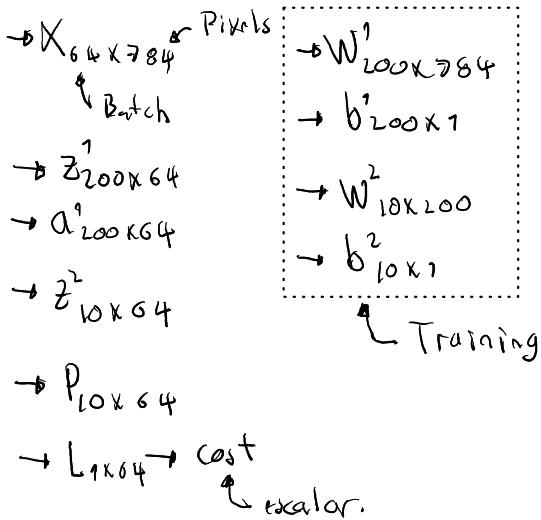
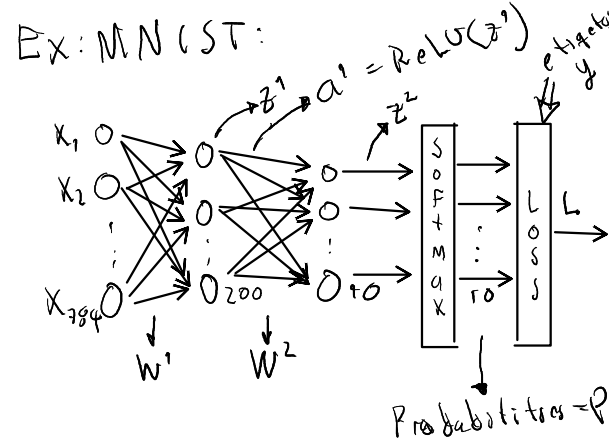
$$\frac{\partial L}{\partial W_{r \times p \times m}^1} = \frac{\partial L_r}{\partial z_q^2} W_{q \times p}^2 \left(\frac{\partial F^1(z^1)}{\partial z^1} \right)_{p \times p} x_m$$

$$\frac{\partial L_r}{\partial z_p^1}$$

$$\rightarrow \frac{\partial L_r}{\partial b_p^1} = \frac{\partial L_r}{\partial z_p^1} \frac{\partial z_p^1}{\partial b_p^1} \hookrightarrow I_{p \times p}$$

$$= \frac{\partial L}{\partial b_{p \times m}^1}$$

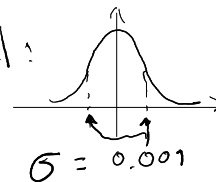
Ex: MNIST:



Inicialmente:

$$W_{200 \times 784}^1 = np.random.randn(200, 784) \cdot 0.001$$

Distribución normal:



$$\rightarrow b_{200 \times 1}^1 = np.zeros((200, 1))$$

$$\rightarrow W_{10 \times 200}^2 = np.random.randn(10, 200) \cdot 0.001$$

$$\rightarrow b_{10 \times 1}^2 = np.zeros((10, 1))$$

Forward:

$$Z_{200 \times 64}^1 = W_{200 \times 784}^1 (X_{64 \times 784})^T + (b_{200 \times 1}^1 \dots b_{200 \times 1}^1)_{200 \times 64}$$

$$\rightarrow a_{200 \times 64}^1 = \text{ReLU}(Z_{200 \times 64}^1)$$

$$\rightarrow Z_{10 \times 64}^2 = W_{10 \times 200}^2 a_{200 \times 64}^1 + (b_{10 \times 1}^2 \dots b_{10 \times 1}^2)_{10 \times 64}$$

Softmax:

$$P_{10 \times 64} = \frac{(e^{Z^2})_{10 \times 64}}{\left(\sum_i e^{Z_{j,i}^2} \right)_{1 \times 64}} = \hat{y}_{10 \times 64} \quad (4.7)$$

suma cada columna (ejm) de $(e^{Z^2})_{10 \times 64}$

Cross entropy:

$$(e^L)_{1 \times 64} = (P_{a1}, P_{a2}, \dots, P_{a64})$$

etiquetas ej. 1 etiquetas ej. 2

$$\rightarrow L_{1 \times 64} = -\ln(e^L)_{1 \times 64}$$

$$\rightarrow \text{cost} = \sum_i L_{1,i} \quad (5.3)$$

Backward:

$$\rightarrow \frac{\partial L}{\partial Z^2}_{10 \times 64} = P_{10 \times 64} - y_{10 \times 64}$$

one hot vector para cada ejemplo:

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 1 \end{pmatrix} \leftarrow \begin{matrix} 1 \\ 2 \\ \vdots \\ 10 \end{matrix}$$

etiquetas

$$\rightarrow \frac{\partial L}{\partial W^2} = \frac{\partial L}{\partial Z^2} \frac{\partial Z^2}{\partial W^2}$$

$$\frac{\partial L}{\partial W^2}_{10 \times 200} = \frac{\frac{\partial L}{\partial Z^2}_{10 \times 64}}{64} (a^1_{200 \times 64})^T$$

$$\rightarrow \frac{\partial L}{\partial b^2} = \frac{\partial L}{\partial Z^2} \frac{\partial Z^2}{\partial b^2}$$

$$\frac{\partial L}{\partial b^2}_{10 \times 1} = \left(\sum_j \frac{\partial L}{\partial Z_{j,1}^2}, \dots, \sum_j \frac{\partial L}{\partial Z_{j,64}^2} \right)_{10 \times 1} / 64$$

sum. elementos de cada row

$$\rightarrow \frac{\partial L}{\partial a^1} = \frac{\partial L}{\partial z^1} \frac{\partial z^1}{\partial a^1}$$

$$\frac{\partial L}{\partial a^1_{200 \times 64}} = (W^2_{10 \times 200})^T \frac{\partial L}{\partial z^1_{10 \times 64}}$$

most be normalized

$$\rightarrow \frac{\partial L}{\partial z^1} = \frac{\partial L}{\partial a^1} \frac{\partial a^1}{\partial z^1}$$

normalized

$\frac{\partial a^1}{\partial z^1_{i,j}}$
 $\delta^1_{z^1_{i,j} > 0}$

$$= \left(\frac{\partial L}{\partial a^1_{i,j}} \delta^1_{z^1_{i,j} > 0} \right)_{200 \times 64}$$

element ij of $\left(\frac{\partial L}{\partial a^1} \right)_{200 \times 64}$

When $z^1_{i,j}$ element > 0 :
 $\delta^1_{z^1_{i,j} > 0} = 1$ (or $z^1_{200 \times 64}$)
 When $z^1_{i,j} \leq 0$:
 $\delta^1_{z^1_{i,j} > 0} = 0$

$$\rightarrow \frac{\partial L}{\partial W^1} = \frac{\partial L}{\partial z^1} \frac{\partial z^1}{\partial W^1}$$

$$\frac{\partial L}{\partial W^1_{200 \times 784}} = \frac{\partial L}{\partial z^1_{200 \times 64}} X_{64 \times 784}$$

most be normalized

$$W^2 = \frac{\partial z^2}{\partial a^1} \rightarrow \frac{\partial L}{\partial a^1}$$

$$\rightarrow \frac{\partial L}{\partial b^1} = \frac{\partial L}{\partial z^1} \frac{\partial z^1}{\partial b^1}$$

$$\frac{\partial L}{\partial b^1_{200 \times 1}} = \left(\sum_j^{64} \frac{\partial L}{\partial z^1_{i,j}} \right)_{200 \times 1}$$

104

Training for each epoch, for each batch:

$$\rightarrow \alpha = 0.009$$

$$\rightarrow W^1_{200 \times 784} = W^1_{200 \times 784} - \alpha \frac{dL}{dW^1_{200 \times 784}}$$

$$\rightarrow b^1_{200 \times 1} = b^1_{200 \times 1} - \alpha \frac{dL}{db^1_{200 \times 1}}$$

$$\rightarrow W^2_{10 \times 200} = W^2_{10 \times 200} - \alpha \frac{dL}{dW^2_{10 \times 200}}$$

$$\rightarrow b^2_{10 \times 1} = b^2_{10 \times 1} - \alpha \frac{dL}{db^2_{10 \times 1}}$$

Accuracy using validation data: after each epoch

Forward (Validation Data)

$$\text{Softmax}(Z^2_{10 \times 64})$$

$$P_{10 \times 64}$$

Prediction = $P_e = (P_{max,1}, \dots, P_{max,64})_{10 \times 64}$

max row of col 1, max row of col 64

Validation data

$$\text{Accuracy} = \frac{\sum_{i=1}^{\# \text{ batches}} \sum_{j=1}^{64} \delta p_{e_j} = \text{Label}_j}{\# \text{ Validation data}}$$

Prediction of a image after

epochs: $\rightarrow$

Optimal $\left\{ \begin{array}{l} \rightarrow W^1_{200 \times 784} \\ \rightarrow b^1_{200 \times 1} \\ \rightarrow W^2_{10 \times 200} \\ \rightarrow b^2_{10 \times 1} \end{array} \right.$

$$\rightarrow \text{image}_{1 \times 784} = X_{1 \times 784}$$

$$\rightarrow z^1_{200 \times 1} = W^1_{200 \times 784} (X_{1 \times 784})^T + b^1_{200 \times 1}$$

$$\rightarrow a^1_{200 \times 1} = \text{ReLU}(z^1_{200 \times 1})$$

$$\rightarrow z^2_{10 \times 1} = W^2_{10 \times 200} a^1_{200 \times 1} + b^2_{10 \times 1}$$

$$\rightarrow P_{10 \times 1} = \frac{(e^{z^2})_{10 \times 1}}{\sum_j e^{z^2_{j,1}}} = \hat{y}_{10 \times 1}$$

Prediction = $P_e = P_{max}$ of $P_{10 \times 1}$

Min. value of pixels in MNIST:
 $\min = 0 \leftarrow \text{Black}$

Max value: (17.2)

$\max = 255 \leftarrow \text{White}$

Normalize pixels to mean=0, std=1:

of every pixels } mean, std
 of every sample }
 of each pixel } data

$$\rightarrow \text{Norm}_{\text{exp}} = \frac{\text{data}_{\text{exp}} - \text{mean}}{\text{std}} \quad (17.3)$$

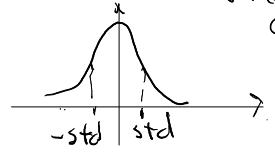
$$\rightarrow \text{Norm}_{\text{mean}}() = 0 \quad (17.4)$$

$$\rightarrow \text{Norm}_{\text{std}}() = 1$$

Matrix $n \times m$ w rand #s by normal distribution, mean 0: (17.1)

$$W_{n \times m} = \text{randn}(n, m) * \text{std}$$

↑
standard deviation



Xavier He init: (17.5)

$$W_{n \times m} = \text{randn}(n, m) \left(\frac{1}{\sqrt{m}} \right)$$

Kaiming init: (17.6)

$$W_{n \times m} = \text{randn}(n, m) \left(\frac{1}{\sqrt{m/2}} \right)$$

Vanishing gradient problem:

Forward:

$$z^1 = W^1 x + b^1$$

$$z^2 = W^2 f(z^1) + b^2$$

$$z^i = W^i f(z^{i-1}) + b^i$$

For normal init:

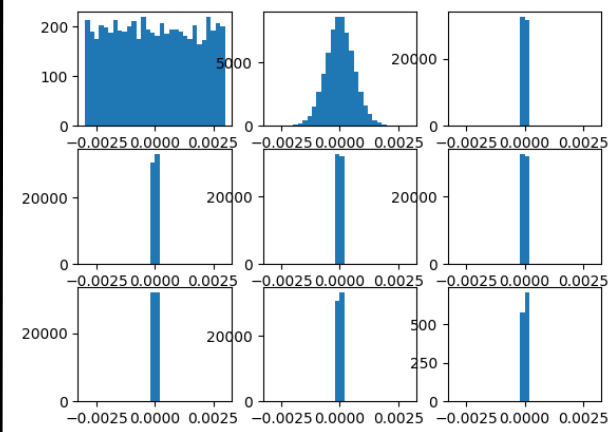
$$z^2 < z^1$$

$$\vdots$$

$$z^i < z^{i-1}$$

$$(17.7)$$

Histograms to $z^1, \dots, z^i$:



So, Backward:

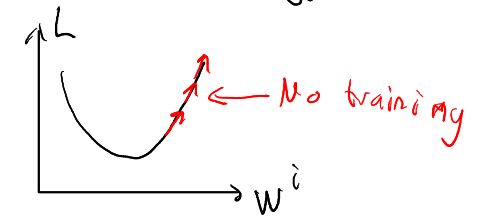
$$\rightarrow \frac{\partial L}{\partial W^i} = \frac{\partial L}{\partial z^i} \underbrace{\frac{\partial z^i}{\partial W^i}}_{\text{Fix} \leftarrow f(z^{i-1})}$$

(17.8)

$$\rightarrow \frac{\partial L}{\partial W^{i-1}} = \frac{\partial L}{\partial z^{i-1}} \underbrace{\frac{\partial z^{i-1}}{\partial W^{i-1}}}_{\text{Fix} \leftarrow f(z^{i-2})}$$

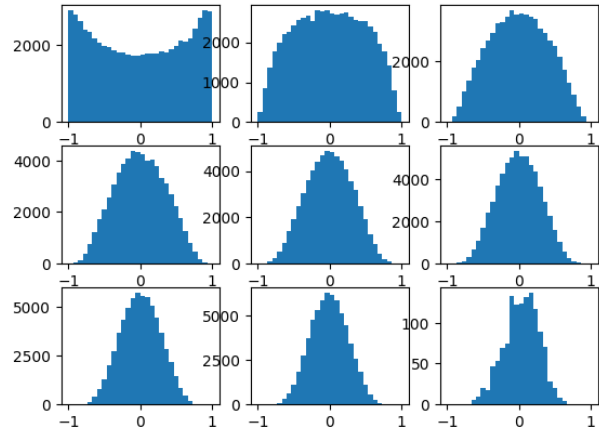
Training:

$$\rightarrow W^i = W^i - \eta \frac{\partial L}{\partial W^i} \quad \text{Fix} \quad (17.9)$$

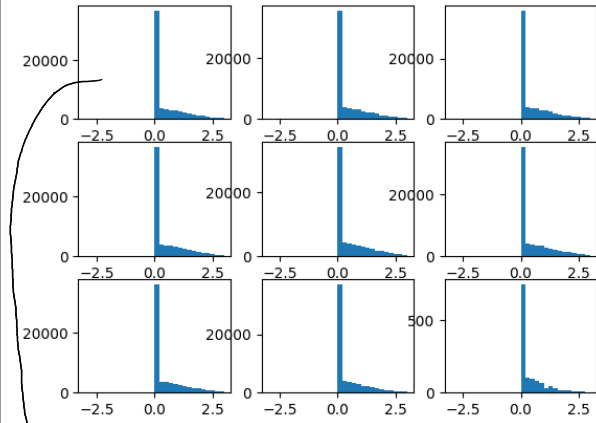


Ex. of No Vanishing problem
using Xavier, $f = \tanh$ and MNIST
↓
X.

Histogram of $z^1, \dots, z^9$:



Ex. of No Vanishing problem
using Kaiming, $f = \text{ReLU}$, MNIST:
Histogram of $z^1, \dots, z^9$:



std = 0.8127

std = 0.894

No Vanishing

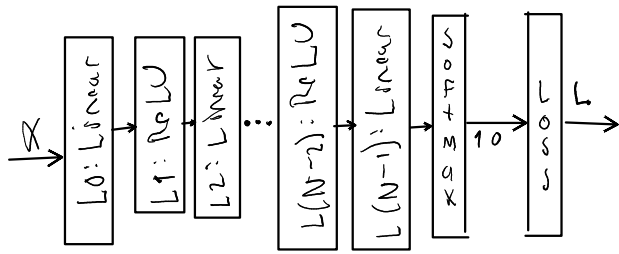
Tips to avoid Vanishing:

Normal $\rightarrow$ Non $p(z)$

Xavier $\rightarrow \tanh(z)$ (17.10)

Kaiming He $\rightarrow \text{ReLU}(z)$
 $\rightarrow \tanh(z)$

Ex. MNIST using POO and sequential layer model:
N layers



Linear: Identity activation function

$\rightarrow X_{b \times 784}$ Pixels
Batch

$\rightarrow Z^0_{0 \times b}$ Linear

$\rightarrow a^1_{0 \times b}$ ReLU

$\rightarrow Z^2_{0 \times b}$ Linear

$\rightarrow a^3_{0 \times b}$ ReLU

$\vdots$

$\rightarrow Z^{N-1}_{0 \times b}$ Linear

$\rightarrow P_{10 \times b}$

$\rightarrow L_{1 \times b} \rightarrow \text{cost}$

escalar

$\rightarrow W^0_{0 \times 784}$
 $\rightarrow b^0_{0 \times 1}$
 $\rightarrow W^2_{0 \times 12}$
 $\rightarrow b^2_{0 \times 1}$
 $\vdots$
 $\rightarrow W^{N-1}_{10 \times i_{N-1}}$
 $\rightarrow b^{N-1}_{10 \times 1}$

Training of W & b of Linear layers

Full connected:

$\rightarrow i_2 = o_0$
Inputs of L2
Outputs of L0

ReLU layers:

$\rightarrow i_1 = o_0$ $\rightarrow i_1 = o_1$

$\rightarrow i_2 = o_2$ $\rightarrow i_3 = o_3$

$\vdots$

$\rightarrow i_{N-2} = o_{N-3}$ $\rightarrow i_{N-1} = o_{N-1}$

Normalization of training, validation and testing images:

$\rightarrow \text{Mean of pixels of training images} = \text{mean}$

$\rightarrow \text{Standard deviation of training images} = \text{std}$

$\rightarrow X_{784} = \frac{X_{784} - \text{mean}}{\text{std}}$ (17.3)

Initialization of W linear layers:

Kawing #0: (17.6)

$W^L_{o_L \times i_L} = \text{randn}(o_L, i_L) \left(\frac{1}{\sqrt{i_L/2}} \right)$

Initialization of b of linear layers:

$b^L_{o_L \times 1} = \text{zeros}((o_L, 1))$

Forward of linear layers:

$\rightarrow Z^0_{0 \times b} = W^0_{0 \times 784} (X_{b \times 784})^T + (b^0 b^0 \dots)_{0 \times b}$

$\vdots$
 $\rightarrow Z^L_{0 \times b} = W^L_{o_L \times i_L} a^{L-1}_{i_L \times b} + (b^L b^L \dots)_{o_L \times b}$
 $i_{L-1} = o_{L-1} = i_L$

L is even

Forward of ReLU layer:

$a^L_{o_L \times b} = \text{ReLU}(Z^L_{o_L \times b})$
 $o_{L-1} = o_L = i_L$

L is odd

Softmax and cross entropy use the last layer L^{N-1} :

$\downarrow$
 $z^{N-1}_{10 \times b}$

For backward:

$\rightarrow z^{N-1} \cdot \text{grad} = \frac{\partial L}{\partial z^{N-1}}$

$\rightarrow W^{N-1} \cdot \text{grad} = \frac{\partial L}{\partial W^{N-1}}$

$= \frac{\partial L}{\partial z^{N-1}} \frac{\partial z^{N-1}}{\partial W^{N-1}}$

$= \frac{\partial L}{\partial z^{N-1}} (a^{N-2})^T$

$\rightarrow b^{N-1} \cdot \text{grad} = \frac{\partial L}{\partial b^{N-1}}$
 $\vdots$

α must be small during learning (6.3)

